Trabaja Práctico. IP. PokeDex

Introducción

Este trabajo presenta una aplicación web diseñada para mostrar tarjetas de Pokémon, cada una con información como su nombre, tipo e imagen, obtenida desde una fuente externa en internet. La aplicación permite a los usuarios ver todas las tarjetas disponibles, buscar Pokémon por su nombre y filtrarlos según su tipo (por ejemplo, fuego, agua o planta). Además, las tarjetas se muestran con colores distintos según su tipo principal, lo que hace que la experiencia sea más atractiva y fácil de entender. El objetivo es ofrecer una herramienta sencilla y visual para que cualquier persona pueda explorar y encontrar Pokémon de manera rápida.
El desarrollo incluyó la creación de funciones para obtener los datos, filtrarlos según los criterios del usuario y mostrarlos en una página web con un diseño claro. A continuación, se presentan las funciones implementadas, su propósito, las dificultades enfrentadas durante el desarrollo y las decisiones tomadas para superarlas.

Funciones Implementadas

1. Función getAllImages

Código:

```
def getAllImages():

    # Obtiene todas las imágenes de pokemones usando transport.getAllImages()
    pokemon_data=transport.getAllImages()
    cartas=[]



    # Itera sobre cada pokemon en la lista de pokemones
    for pokemon in pokemon_data:

        # Agrega cada pokemon a la lista de cartas
        #Llamo a la función para convertir el json en una card
        cartas.append(fromRequestIntoCard(pokemon))
```

```
    # Retorna la lista completa de cartas después de procesar todos los
pokemones
    return cartas
```

Explicación: Esta función obtiene una lista de todos los Pokémon disponibles desde una fuente externa (una API en internet) y los convierte en tarjetas que la aplicación puede usar. Cada tarjeta incluye detalles como el nombre, los tipos y la imagen del Pokémon. La función recorre los datos obtenidos y los transforma en objetos que luego se muestran en la página web.

Dificultades de implementación:

- **Dependencia de una función externa**: La función depende de transport.getAllImages() y fromRequestIntoCard, que no fueron desarrolladas por nosotros, lo que limitó nuestro control sobre posibles fallos.

Decisiones tomadas y justificación:

- **Usar un bucle simple**: Se decidió usar un bucle para procesar los datos uno por uno, ya que era una forma clara y directa de garantizar que cada Pokémon se convirtiera en una tarjeta. Esto facilitó la detección de errores en los datos.
- **No modificar funciones externas**: Como fromRequestIntoCard ya estaba proporcionada, se optó por confiar en su correcto funcionamiento para transformar los datos, evitando duplicar trabajo o introducir errores al intentar modificarla.

2. Función filterByCharacter

Código:

```
def filterByCharacter(name):
    # Define una función que acepta un parámetro 'name' para filtrar tarjetas
según su nombre.
    filtered_cards = []
    # Inicializa una lista vacía para almacenar las tarjetas que coincidan con
el criterio de búsqueda.
    for card in getAllImages():
        # Itera sobre cada objeto Card devuelto por la función getAllImages.
        if card.name and name.lower() in card.name.lower():
            # Verifica si el atributo 'name' de la tarjeta existe y si el
parámetro 'name' (en minúsculas) está contenido en el nombre de la tarjeta
(también en minúsculas).
            filtered_cards.append(card)
            # Añade la tarjeta a la lista 'filtered_cards' si la condición de
búsqueda se cumple.
```

```
    return filtered_cards
    # Devuelve la lista 'filtered_cards' con todas las tarjetas cuyo atributo
'name' contiene el parámetro 'name'.
```

Explicación: Esta función permite buscar Pokémon por su nombre. El usuario ingresa una palabra (por ejemplo, "pika") y la función devuelve todas las tarjetas cuyo nombre contenga esa palabra, sin importar si está escrita en mayúsculas o minúsculas. Esto hace que la búsqueda sea flexible y fácil de usar.

Dificultades de implementación:

- **Sensibilidad a mayúsculas**: Los usuarios podían escribir la búsqueda en mayúsculas o minúsculas, lo que requería un manejo especial para que la función no fallara.

Decisiones tomadas y justificación:

- **Normalizar la entrada**: Se decidió convertir tanto el nombre buscado como el nombre de la tarjeta a minúsculas usando name.lower() y card.name.lower(). Esto garantiza que la búsqueda no dependa de cómo el usuario escriba la palabra, mejorando la experiencia.
- **Entrada vacía:** Se decidió que cuando un usuario, utilice el buscador sin ninguna entrada, devolviera todas las tarjetas.
- **Entrada errónea:** Si no se encuentran coincidencias con el dato ingresado la página devolvera el siguiente mensaje: “La búsqueda no arrojo resultados...”

3. Función filterByType

Código:

```
def filterByType(type_filter):
    # Define una función que acepta un parámetro 'type_filter' para filtrar
tarjetas según su tipo.
    filtered_cards = []
    # Normaliza el parámetro 'type_filter' convirtiéndolo a minúsculas y
eliminando espacios.
    type_filter = type_filter.lower().strip()
    # Si la longitud del parámetro 'type_filter' es cero, retorna una lista
vacía.
    if len(type_filter) > 0:
        for card in getAllImages():
            # Verifica si el atributo 'type' de la tarjeta existe y si
'type_filter' está contenido en él.
            if card.types and any(type_filter in str(item).lower() for item in
card.types):
                # Añade la tarjeta a la lista 'filtered_cards' si el tipo
coincide.
                filtered_cards.append(card)

    return filtered_cards
```

```
    # Devuelve la lista 'filtered_cards' con todas las tarjetas cuyo atributo
'type' contiene el parámetro 'type_filter'.
```

Explicación: Esta función filtra las tarjetas según el tipo de Pokémon (por ejemplo, "fuego"). En la aplicación el usuario se encontrará con 3 botones (fuego, agua y planta), cuado alguno de esos tres botones es clickeado, la función devuelve todas las tarjetas que tengan ese tipo en su lista.

Dificultades de implementación:

- **Tipos múltiples**: Algunos Pokémon tienen más de un tipo, lo que requería verificar cada tipo de la tarjeta.

Decisiones tomadas y justificación:

- **Uso de** any **para múltiples tipos**: Se usó any para comprobar si el tipo buscado está en la lista de tipos de la tarjeta, permitiendo manejar Pokémon con varios tipos de manera eficiente.
- **Conversión a cadenas**: Se decidió convertir cada tipo a cadena (str(item)) para garantizar compatibilidad.

Código de Vistas y Plantilla

Vistas (views.py)

Código relevante:

```
# esta función obtiene 2 listados: uno de las imágenes de la API y otro de
favoritos, ambos en formato Card, y los dibuja en el template 'home.html'.
def home(request):

    #importamos la función getAllImages de services para mostrar las cards
    images=getAllImages()

    #armo una lista vacía
    favourite_list = []

    #devuelve las cards a la lista vacía
    return render(request, 'home.html', { 'images': images, 'favourite_list':
favourite_list })
```

```
# función utilizada en el buscador.
def search(request):
    name = request.POST.get('query', '')
```

```
    # si el usuario ingresó algo en el buscador, se deben filtrar las imágenes
por dicho ingreso.
    if (name != ''):
        images = filterByCharacter(name)
        favourite_list = []

        return render(request, 'home.html', { 'images': images,
'favourite_list': favourite_list })
    else:
        return redirect('home')
# función utilizada para filtrar por el tipo del Pokemon
def filter_by_type(request):
    type = request.POST.get('type', '')

    if type != '':
        images = filterByType(type)
        favourite_list = []

        return render(request, 'home.html', { 'images': images,
'favourite_list': favourite_list })
    else:
        return redirect('home')
```

Explicación:

- home: Muestra todas las tarjetas de Pokémon en la página principal.
- search: Filtra las tarjetas por nombre según lo que el usuario escribe en un formulario.
- filter_by_type: Filtra las tarjetas por tipo según la selección del usuario.

Plantilla (home.html)

Código relevante:

```
  <div class="card {% if img.types|first == 'grass' %}border-success{% elif
img.types|first == 'fire' %}border-danger{% elif img.types|first == 'water'
%}border-primary{% elif img.types|first == 'electric' %}border-warning{% else
%}border-secondary{% endif %}">
```

Explicación: Esta parte de la plantilla muestra cada tarjeta de Pokémon con un borde de color según su tipo principal (por ejemplo, verde para planta, rojo para fuego). Si el tipo no está entre los definidos, se usa un borde gris.

Dificultades de implementación:

- **Tipos no soportados**: Solo se definieron colores para algunos tipos, lo que hacía que Pokémon con otros tipos (como poison) usaran el borde gris.
- **Acceso al primer tipo**: Era necesario asegurarse de que img.types fuera una lista válida para usar el filtro |first.

Decisiones tomadas y justificación:

- **Colores limitados**: Se decidió soportar solo los tipos más comunes (grass, fire, water, electric) para simplificar el diseño inicial, usando un borde gris para otros tipos como solución genérica.
- **Uso de** |first: Se optó por usar el primer tipo de la lista para definir el color, ya que es una forma sencilla de asignar un estilo visual sin complicar la lógica.

Plantilla (header.html)

Código relevante:

```
<a class="navbar-brand"
href="https://github.com/juliangiardelli/Pokedex_julian_giardelli.git">Proyect
o TP | Julián Giardelli</a>
```

Explicación: Esta parte de la plantilla se modifico el href, con mi link al repositorio de GitHub para que te envíe a mi repositorio. Además, dentro de la etiqueta puse mi nombre completo para identificar quien ha realizado el proyecto.

Estilos CSS

Código:

```
.border-success {
    border-color: #28a745; /* Verde para planta */
}

.border-danger {
    border-color: #ff0000; /* Rojo para fuego*/
}

.border-primary {
    border-color: #00bfff; /* Azul agua */
}

.border-secondary {
    border-color: #808080; /* Gris para normal */
}

.border-warning{
    border-color: #f6ff00; /* Amarillo electrico*/
```

```
}
```

Explicación: Define los colores de los bordes de las tarjetas según el tipo del Pokémon, haciendo que la página sea visualmente atractiva.
Dificultades de implementación:

- **Error tipográfico**: La clase .border-warning estaba mal escrita, lo que impedía que el estilo para Pokémon eléctricos se aplicara.
- **Limitación de tipos**: Solo se definieron estilos para cinco tipos, dejando otros sin un color específico.

Decisiones tomadas y justificación:

- **Colores intuitivos**: Se eligieron colores que representaran claramente cada tipo (por ejemplo, rojo para fuego, azul para agua) para que los usuarios asociaran fácilmente el color con el tipo.
- **Uso de gris por defecto**: Se decidió usar un borde gris (border-secondary) para tipos no soportados, como una solución simple para evitar errores visuales.

Conclusión

La aplicación desarrollada permite a los usuarios explorar, buscar y filtrar Pokémon de manera sencilla, mostrando las tarjetas con un diseño atractivo gracias a los colores basados en los tipos. Las funciones implementadas trabajan juntas para obtener datos, procesarlos y mostrarlos en una página web. Aunque se enfrentaron desafíos como datos inconsistentes y limitaciones en los estilos, las decisiones tomadas priorizaron la simplicidad y la experiencia del usuario. Para futuras mejoras, se podría soportar más tipos de Pokémon y mejorar el manejo de errores en los datos de la API.